

UNIMA — UNIVERSIDADE DE MACEIÓ
CURSO DE CIÊNCIA DA COMPUTAÇÃO

IVANILDO MARQUES DE SOUZA FILHO

**RACHACONTAS: SISTEMA DE FINANÇAS COLETIVAS PARA REPÚBLICAS E
GRUPOS**

Relatório Técnico

Relatório técnico entregue à disciplina de Projeto de Programação do Curso de Ciência da
Computação, sob orientação do Prof. Amaury Nogueira.

2026

IVANILDO MARQUES DE SOUZA FILHO

Matrícula: 1211539404

**RACHACONTAS: SISTEMA DE FINANÇAS COLETIVAS PARA REPÚBLICAS E
GRUPOS**

Relatório Técnico

Relatório técnico apresentado como requisito
parcial da disciplina de Projeto de Programação
do Curso de Ciência da Computação. Professor:
Amaury Nogueira.

2026

SUMÁRIO

1	INTRODUÇÃO	4
1.1	Objetivos	4
1.2	Tecnologias Utilizadas	4
2	METODOLOGIA DE IMPLEMENTAÇÃO	5
2.1	Arquitetura — Domain-Driven Design (DDD)	5
2.2	Estruturas de Dados Implementadas	5
2.3	Algoritmo de Acertos (Greedy Clearing)	7
2.4	Test-Driven Development (TDD)	7
2.5	Integração Contínua — GitHub Actions	8
2.6	Documentação de Setup	8
3	CASOS DE TESTE	9

1 INTRODUÇÃO

O presente relatório descreve o desenvolvimento do sistema denominado *RachaConta*, um aplicativo de finanças coletivas destinado a repúblicas estudantis e grupos de pessoas que compartilham despesas cotidianas. O sistema foi construído como projeto da disciplina de Projeto de Programação do Curso de Ciência da Computação, correspondendo ao Tema 10 da lista de temas propostos.

O problema abordado consiste na dificuldade, comum em ambientes coletivos como repúblicas, de controlar despesas compartilhadas. Em tais contextos, cada membro do grupo realiza pagamentos em momentos distintos — contas de água, luz, internet, mercado — e ao final do período há dificuldade em determinar com precisão quem deve quanto a quem. O sistema resolve esse problema registrando os gastos, distribuindo as despesas proporcionalmente e calculando os acertos mínimos necessários para quitar todos os saldos.

A implementação aplicou os princípios de Orientação a Objetos avançada, Domain-Driven Design (DDD), Test-Driven Development (TDD) e integração contínua via GitHub Actions, conforme exigido pela rubrica de avaliação.

1.1 Objetivos

- a) Registrar despesas compartilhadas entre membros de um grupo;
- Calcular automaticamente os saldos de cada membro (credor ou devedor);
- Determinar o conjunto mínimo de transferências para zerar todas as dívidas;
- Disponibilizar interface web para uso intuitivo pelo usuário final;
- Garantir a correção das regras de negócio por meio de testes automatizados.

1.2 Tecnologias Utilizadas

Tecnologia	Versão	Finalidade
TypeScript	5.x	Linguagem principal de desenvolvimento
Node.js	20.x	Ambiente de execução
Express	4.18	Framework para o servidor web (API REST)
Jest	29.x	Framework de testes unitários

GitHub Actions	—	Pipeline de integração contínua (CI/CD)
NPM	10.x	Gerenciador de dependências

Quadro 1 — Tecnologias utilizadas no projeto.

2 METODOLOGIA DE IMPLEMENTAÇÃO

Esta seção descreve as decisões arquiteturais, as estruturas de dados implementadas, o processo de desenvolvimento orientado a testes e a configuração da esteira de integração contínua.

2.1 Arquitetura — Domain-Driven Design (DDD)

O projeto foi estruturado segundo os princípios do Domain-Driven Design (EVANS, 2003), separando o código em quatro camadas com responsabilidades bem definidas, conforme o Quadro 2.

Camada	Diretório	Responsabilidade
Domain	src/domain/	Entidades, Value Objects, Aggregates — regras de negócio puras
Application	src/application/	Casos de uso e orquestração dos fluxos da aplicação
Infrastructure	src/infrastructure/	Repositórios, persistência de dados e adaptadores externos
Presentation	src/presentation/	Servidor web (API REST) e interface gráfica (HTML)

Quadro 2 — Estrutura de camadas da arquitetura DDD.

Essa separação garante que as regras de negócio, concentradas na camada *domain*, sejam independentes de detalhes técnicos como banco de dados ou framework web. Assim, é possível substituir a tecnologia de persistência sem alterar nenhuma linha das regras do domínio.

2.2 Estruturas de Dados Implementadas

2.2.1 Value Object — Dinheiro

O Value Object *Dinheiro* representa valores monetários de forma segura e imutável. A principal decisão de projeto foi armazenar o valor internamente em **centavos** (números inteiros), evitando o erro clássico de arredondamento do padrão IEEE 754, no qual

operações com ponto flutuante como $0,1 + 0,2$ resultam em $0,30000000000000004$. A imutabilidade é garantida pelo modificador *readonly* e pelo retorno de novos objetos em todas as operações aritméticas.

```
// src/domain/value-objects/Dinheiro.ts (trecho)
export class Dinheiro {
  private readonly _centavos: number;

  static deReais(reais: number): Dinheiro {
    const centavos = Math.round(reais * 100); // R$ 10,50 => 1050
    return new Dinheiro(centavos);
  }

  somar(outro: Dinheiro): Dinheiro {
    return new Dinheiro(this._centavos + outro._centavos); // novo objeto
  }

  dividir(quantidade: number): Dinheiro {
    return new Dinheiro(Math.ceil(this._centavos / quantidade));
  }
}
```

Código 1 — Value Object Dinheiro (trecho).

2.2.2 Entidade — *Membro*

A entidade *Membro* representa cada participante do grupo. Possui identidade própria definida por um UUID gerado automaticamente. O saldo é mantido internamente em centavos e pode assumir valores positivos (indicando que o membro é credor) ou negativos (indicando que é devedor). Os atributos são todos privados, acessíveis apenas por métodos controlados, aplicando encapsulamento rigoroso.

2.2.3 Entidade — *Despesa*

A entidade *Despesa* registra um gasto do grupo, armazenando: descrição, valor (como Value Object *Dinheiro*), identificador de quem pagou, lista de membros que participam da divisão e categoria (mercado, água, luz, internet, aluguel ou outros). O valor por pessoa é calculado pelo método *dividir* do Value Object, garantindo arredondamento correto.

2.2.4 Aggregate Root — *GrupoFinanceiro*

O *GrupoFinanceiro* é a classe raiz do agregado principal. Nenhum membro ou despesa pode ser criado diretamente — toda interação ocorre por meio do *GrupoFinanceiro*, que centraliza as validações de consistência: impede membros com nomes duplicados, rejeita despesas referenciando membros inexistentes e proíbe a remoção de membros que possuam despesas vinculadas.

```
// Exemplo de uso do Aggregate Root
const grupo = new GrupoFinanceiro("República das Meninas");
const ana = grupo.adicionarMembro(new NomeMembro("Ana"));
const bia = grupo.adicionarMembro(new NomeMembro("Bia"));

grupo.registrarDespesa(new Despesa({
  descricao: "Conta de luz",
  valor: Dinheiro.deReais(120.00),
  quemPagouId: ana.id,
  dividirComIds: [ana.id, bia.id],
  categoria: CategoriaDespesa.LUZ
})));
// ana.saldoCentavos => +6000 (credora: R$ 60,00)
// bia.saldoCentavos => -6000 (devedora: R$ 60,00)
```

Código 2 — Exemplo de uso do Aggregate Root.

2.3 Algoritmo de Acertos (Greedy Clearing)

Para calcular o conjunto mínimo de transferências necessárias para quitar todos os saldos, foi implementado o algoritmo *Greedy Clearing*. O algoritmo ordena os devedores e credores pelo valor absoluto do saldo (do maior para o menor) e realiza transferências iterativas até que todos os saldos sejam zerados.

```
// Lógica do algoritmo Greedy Clearing (simplificado)
const devedores = membros.filter(m => m.ehDevedor).sort(maiorPrimeiro);
const credores = membros.filter(m => m.ehCredor).sort(maiorPrimeiro);

while (i < devedores.length && j < credores.length) {
  const transferir = Math.min(devedores[i].saldo, credores[j].saldo);
  acertos.push({ de: devedores[i], para: credores[j], valor: transferir });
  devedores[i].saldo -= transferir;
  credores[j].saldo -= transferir;
  if (devedores[i].saldo === 0) i++;
  if (credores[j].saldo === 0) j++;
}
```

Código 3 — Algoritmo Greedy Clearing (trecho).

2.4 Test-Driven Development (TDD)

O desenvolvimento seguiu o ciclo *Red-Green-Refactor* do TDD: os testes foram escritos antes da implementação das classes de domínio. Ao todo foram criados 3 arquivos de teste, cobrindo as três classes principais do domínio com 31 casos de teste, sendo cada um voltado tanto para cenários de sucesso quanto para cenários de falha (validação de regras de negócio).

Arquivo de Teste	Classe Testada	Casos de Teste
tests/domain/Dinheiro.test.ts	Dinheiro (Value Object)	13
tests/domain/Despesa.test.ts	Despesa (Entidade)	8
tests/domain/GrupoFinanceiro.test.ts	GrupoFinanceiro (Aggregate)	10
Total		31

Quadro 3 — Arquivos de teste e cobertura por classe.

2.5 Integração Contínua — GitHub Actions

O arquivo `.github/workflows/ci.yml` configura a esteira de integração contínua. A cada *push* ou abertura de *pull request* na branch *main*, o pipeline executa automaticamente: instalação de dependências (*npm ci*), compilação TypeScript (*npm run build*) e execução de todos os testes (*npm test*). O *build* é considerado aprovado somente quando 100% dos testes passam.

2.6 Documentação de Setup

O arquivo *README.md* contém os comandos exatos necessários para instalar as dependências, executar os testes e iniciar o servidor web:

```
# Pré-requisito: Node.js (versão LTS) instalado – nodejs.org

# 1. Acessar a pasta do projeto
cd poo-pbl-grupo-republica

# 2. Instalar as dependências
npm install

# 3. Executar os testes unitários
npm test

# 4. Iniciar o servidor web
npm run dev

# 5. Acessar a interface no navegador
```



```
http://localhost:3000
```

Código 4 — Comandos de setup do projeto.

3 CASOS DE TESTE

Esta seção apresenta os principais casos de teste implementados no projeto, demonstrando as entradas fornecidas ao sistema, o comportamento esperado segundo as regras de negócio e as saídas produzidas pela execução dos testes.

3.1 CT-01 — Operações Aritméticas com Value Object Dinheiro

Objetivo: verificar que operações monetárias não produzem erros de arredondamento decorrentes do uso de ponto flutuante.

Campo	Descrição
Entrada	<code>new Dinheiro(1000).somar(new Dinheiro(30))</code>
Esperado	Objeto Dinheiro com <code>_centavos = 1030</code> (R\$ 10,30)
Resultado	PASSOU ✓

Quadro 4 — CT-01: operações aritméticas.

```
// tests/domain/Dinheiro.test.ts
test("deve somar dois valores corretamente", () => {
  const a = new Dinheiro(1000); // R$ 10,00
  const b = new Dinheiro(30);   // R$ 0,30
  const resultado = a.somar(b);
  expect(resultado.centavos).toBe(1030); // PASSOU
});

// Saída do Jest:
// PASS tests/domain/Dinheiro.test.ts
//   Dinheiro - Value Object > operações aritméticas
//     ✓ deve somar dois valores corretamente (2 ms)
```

Código 5 — CT-01: código e saída do teste.

3.2 CT-02 — Rejeição de Valor Monetário Negativo

Objetivo: garantir que o sistema não permite a criação de um objeto *Dinheiro* com valor negativo, validando a regra de negócio.

```
// tests/domain/Dinheiro.test.ts
test("valor for negativo deve lançar exceção", () => {
  expect(() => new Dinheiro(-1)).toThrow("negativo");
});
```

```
// Saída:
//   ✓ valor for negativo deve lançar exceção (1 ms)
```

Código 6 — CT-02: rejeição de valor negativo.

3.3 CT-03 — Atualização de Saldos ao Registrar Despesa

Objetivo: verificar que ao registrar uma despesa paga por Ana e dividida entre Ana e Bia, os saldos são calculados e atualizados corretamente.

Campo	Descrição
Entrada	Ana paga R\$ 100,00 dividir entre: [Ana, Bia]
Esperado	Ana: saldo +5000 centavos (+R\$ 50,00) Bia: saldo -5000 centavos (-R\$ 50,00)
Resultado	PASSOU ✓

Quadro 5 — CT-03: atualização de saldos.

```
// tests/domain/GrupoFinanceiro.test.ts
test("deve registrar despesa e atualizar saldos corretamente", () => {
  const { grupo, ana, bia } = criarGrupoComMembros();

  grupo.registrarDespesa(new Despesa({
    descricao: "Mercado",
    valor: Dinheiro.deReais(100),
    quemPagouId: ana.id,
    dividirComIds: [ana.id, bia.id]
  }));

  expect(ana.saldoCentavos).toBe(5000); // +R$ 50,00 - PASSOU
  expect(bia.saldoCentavos).toBe(-5000); // -R$ 50,00 - PASSOU
});

// Saída:
//   ✓ deve registrar despesa e atualizar saldos corretamente (3 ms)
```

Código 7 — CT-03: código e saída do teste.

3.4 CT-04 — Cálculo de Acertos com Três Membros

Objetivo: verificar que o algoritmo Greedy Clearing produz o conjunto correto de transferências em um cenário com três membros.

```
// tests/domain/GrupoFinanceiro.test.ts
test("cenário com 3 pessoas e múltiplas despesas", () => {
  // Ana paga R$ 90,00 dividido entre Ana, Bia e Carol (R$ 30 cada)
  // Saldos resultantes: Ana +6000, Bia -3000, Carol -3000
```

```

grupo.registrarDespesa(new Despesa({
  valor: Dinheiro.deReais(90),
  quemPagouId: ana.id,
  dividirComIds: [ana.id, bia.id, carol.id]
}));

const acertos = grupo.calcularAcertos();
const totalAcertos = acertos.reduce((s, a) => s + a.valor.centavos, 0);
expect(totalAcertos).toBe(6000); // R$ 60,00 – PASSOU
});

// Saída:
//   ✓ cenário com 3 pessoas e múltiplas despesas (4 ms)

```

Código 8 — CT-04: cálculo de acertos com três membros.

3.5 CT-05 — Regra de Negócio: Impedir Remoção de Membro com Despesas

Objetivo: verificar que o Aggregate Root impede a remoção de um membro que possua despesas vinculadas, preservando a consistência dos dados.

```

// tests/domain/GrupoFinanceiro.test.ts
test("não deve remover membro com despesas", () => {
  grupo.registrarDespesa(new Despesa({
    descricao: "Mercado",
    valor: Dinheiro.deReais(50),
    quemPagouId: ana.id,
    dividirComIds: [ana.id, bia.id]
  }));

  expect(() => grupo.removerMembro(ana.id)).toThrow("despesas"); // PASSOU
});

// Saída:
//   ✓ não deve remover membro com despesas (1 ms)

```

Código 9 — CT-05: proteção de integridade do agregado.

3.6 Resultado Geral da Execução dos Testes

A execução completa da suíte de testes com o comando *npm test* produz o seguinte resultado:

```

$ npm test

> jest --coverage

PASS tests/domain/Dinheiro.test.ts
PASS tests/domain/Despesa.test.ts

```

```
PASS tests/domain/GrupoFinanceiro.test.ts
```

```
Test Suites: 3 passed, 3 total  
Tests:      31 passed, 31 total  
Time:       2.847 s
```

File	% Stmts	% Branch	% Funcs	% Lines
Dinheiro	100,00	100,00	100,00	100,00
Despesa	97,50	90,00	100,00	97,50
GrupoFin.	95,20	88,00	100,00	95,20

Código 10 — Saída completa da execução dos testes.

REFERÊNCIAS

EVANS, Eric. **Domain-Driven Design: Tackling Complexity in the Heart of Software**. Boston: Addison-Wesley, 2003.

BECK, Kent. **Test Driven Development: By Example**. Boston: Addison-Wesley, 2002.

MARTIN, Robert C. **Clean Architecture: A Craftsman's Guide to Software Structure and Design**. Boston: Prentice Hall, 2017.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 14724: Informação e documentação — Trabalhos acadêmicos — Apresentação**. Rio de Janeiro: ABNT, 2011.

MICROSOFT. **TypeScript Documentation**. Disponível em: <https://www.typescriptlang.org/docs>. Acesso em: jun. 2026.